Stable Diffusion WebUI

AUTOMATIC1111 — Technical Overview & Implementation Guide

Python | PyTorch | Gradio | FastAPI | Stable Diffusion (SD1.x / SD2.x / SDXL / SD3)

1. Project Overview

Stable Diffusion WebUI (commonly known as AUTOMATIC1111 after its creator) is an open-source, feature-rich web interface for running Stable Diffusion image generation models locally. Built on the Gradio framework with a FastAPI backend, it provides a browser-based UI that covers the full spectrum of AI image generation workflows — from simple text-to-image generation to advanced inpainting, outpainting, upscaling, training, and API access. It is one of the most widely used local AI image generation front-ends in existence.

1.1 Core Capabilities

- Text-to-Image (txt2img): Generate images from text prompts with full control over sampling, resolution, batch size, seeds, CFG scale, and styles.
- Image-to-Image (img2img): Transform existing images using a prompt and denoising strength; supports five sub-modes: standard img2img, sketch, inpaint, inpaint sketch, and inpaint with uploaded mask.
- Inpainting & Outpainting: Paint over areas of an image to regenerate them (inpaint), or extend an image beyond its borders (outpaint) using built-in scripts.
- Highres Fix: Two-pass generation pipeline that first renders at low resolution, then upscales and refines at high resolution — preventing the distortions common in one-pass high-res generation.
- Extras / Post-processing: Dedicated tab for upscaling (RealESRGAN, ESRGAN, SwinIR, LDSR), face restoration (GFPGAN, CodeFormer), and batch image processing.
- Checkpoint Merger: Merge up to three Stable Diffusion checkpoints with configurable weights and interpolation methods directly in the UI.
- Textual Inversion Training: Train custom embeddings (concepts) on a set of reference images from within the UI.
- Prompt editing & scheduling: Change prompts mid-generation ([word1:word2:0.5] syntax), use attention weights ((word:1.2)), alternate prompts ([word1|word2]), and compose prompts with AND.
- X/Y/Z Grid plots: Run systematic experiments across multiple parameter values and visualize results as a grid.
- REST API: A full FastAPI-based API that mirrors every UI capability, enabling automation and integration with external tools.
- Extension system: A plugin architecture allowing community extensions to add new tabs, scripts, processing hooks, and UI components without modifying core code.
- Multi-platform & multi-hardware: Native support for NVIDIA CUDA, AMD (ROCm), Intel XPU (IPEX), Ascend NPU, Apple MPS (via macOS script), and CPU-only fallback.

1.2 Technology Stack

Layer	Technology	Version / Notes
Web UI Framework	Gradio	3.41.2 — browser-based interactive UI
API Framework	FastAPI + Uvicorn	0.94.0 — REST API at /docs and /redoc

Deep Learning	PyTorch	>=2.0 recommended; CUDA/ROCm/MPS/XPU
Model Acceleration	xformers (optional)	Cross-attention memory optimization
SD Base Libraries	LDM + SGM	latent-diffusion (CompVis) + stability-sdk
CLIP / Text Encoders	transformers	4.30.2 — HuggingFace transformers
Open CLIP	open-clip-torch	2.20.0 — for SD2.x and SDXL text encoding
Image Processing	Pillow + OpenCV	9.5.0 / scikit-image 0.21.0
Checkpoint Format	safetensors	0.4.2 — safe model loading (also .ckpt via pickle)
Config Management	OmegaConf	2.2.3 — YAML model configs
Upscaling Models	spandrel	0.3.4 — unified upscaler model loader
Memory Profiling	psutil + tomesd	Token merging for speed/VRAM tradeoff
Version Control	GitPython	3.1.32 — extension management via git
Caching	diskcache	5.6.3 — persistent disk-based cache
Launch / Setup	launch.py	Auto-installs dependencies, clones repos, sets up environment

2. System Architecture

2.1 Startup & Launch Pipeline

The application has a two-stage launch system. The outer launcher (launch.py / webui.sh / webui.bat) handles environment bootstrapping — installing Python packages, cloning required repositories (LDM, k-diffusion, BLIP, etc.), and verifying system requirements — before handing off to the inner application (webui.py).

Stage	File	Responsibility
1. OS Launch Script	webui.sh / webui.bat / webui-user.sh	Sets COMMANDLINE_ARGS, activates venv, calls launch.py
2. Environment Setup	launch.py → modules/launch_utils.py	pip installs requirements, git-clones repos, version checks
3. App Entry	webui.py → webui() or api_only()	Imports, version check, calls initialize.initialize()
4. Initialization	modules/initialize.py	Imports torch/gradio, sets up paths, loads SD model, samplers, scripts, upscalers, VAEs, extensions
5. UI Creation	modules/ui.py → create_ui()	Builds all Gradio tabs and components
6. Server Launch	gradio.Blocks.launch()	Starts Uvicorn server; Gradio queue holds 64 concurrent jobs
7. Hot Reload	shared.state.wait_for_server_command()	Supports restart without full process kill; reloads scripts and extensions

2.2 Module Layout

All application logic lives in the modules/ package. The project separates UI code (modules/ui*.py), processing pipelines (modules/processing.py, txt2img.py, img2img.py), model management (modules/sd_models.py, sd_vae.py), and infrastructure (shared.py, devices.py, paths.py).

Module / Path	Purpose
webui.py	Entry point: webui() loop with hot-reload, api_only() for headless API
launch.py + modules/launch_utils.py	Environment bootstrap: pip, git, environment validation
modules/initialize.py	Ordered startup: imports → model setup → samplers → scripts → upscalers → VAE → extensions
modules/shared.py + shared_*.py	Global state: opts (settings), cmd_opts (CLI args), sd_model reference, device, state machine

modules/cmd_args.py	100+ argparse arguments covering hardware, paths, API, UI, security, and performance
modules/processing.py	Core diffusion pipeline: StableDiffusionProcessingTxt2Img / Img2Img classes, process_images()
modules/txt2img.py	txt2img handler: builds processing object, runs scripts, calls process_images()
modules/img2img.py	img2img handler: resolves 5 input modes (img2img/sketch/inpaint/inpaint-sketch/batch), process_batch()
modules/sd_models.py	Checkpoint management: discovery, loading (.ckpt/.safetensors), ModelType enum (SD1/SD2/SDXL/SSD/SD3)
modules/sd_vae.py	VAE management: separate VAE loading, hot-swapping, hash caching
modules/sd_samplers*.py	Sampler registry: k-diffusion, CompVis DDIM/PLMS, LCM samplers; unified SamplerData interface
modules/sd_schedulers.py	Noise schedulers (separate from samplers): linear, cosine, karras, etc.
modules/sd_hijack*.py	Model monkey-patching: CLIP hijack, cross-attention optimizations, VRAM management hooks
modules/prompt_parser.py	Lark grammar parser: attention weights, scheduled swaps, alternating prompts, AND composition
modules/scripts.py	Script/extension base classes: Script, AlwaysVisible; loading and execution pipeline
modules/script_callbacks.py	Event hook system: 30+ named callbacks for extensions to intercept processing stages
modules/upscaler.py + *_model.py	Abstract Upscaler base class; concrete implementations for RealESRGAN, ESRGAN, SwinIR, DAT, HAT, LDSR
modules/face_restoration*.py	Abstract face restoration + utils; GFPGAN and CodeFormer implementations
modules/images.py	Image I/O: save with metadata (PNG chunks / EXIF), grid creation, resize, watermark, AVIF support
modules/extras.py	Extras tab backend: upscale + face restore pipeline, batch processing
modules/ui.py	Main Gradio UI builder: all tabs wired to backend functions
modules/ui_extra_networks.py	Extra Networks panel: Checkpoints, Lora, Hypernetworks, Textual Inversion browsers
modules/extensions.py	Extension discovery, loading order, git update support
modules/api/api.py	FastAPI router: REST endpoints mirroring all UI functions
modules/devices.py	Hardware abstraction: CUDA/MPS/XPU/NPU device selection, dtype management

modules/lowvram.py	Low-VRAM optimization: module-level CPU offloading (--lowvram / --medvram)
modules/rng.py + rng_philox.py	Deterministic RNG: seed management, Philox GPU RNG for reproducibility across hardware
modules/safe.py	Safe unpickle: blocks malicious code in .ckpt files
scripts/	Built-in scripts: xyz_grid, prompt_matrix, loopback, outpainting, img2imgalt, prompts_from_file, sd_upscale

3. Core Processing Pipeline

3.1 Txt2Img Pipeline

Text-to-image generation is handled by the StableDiffusionProcessingTxt2Img class in modules/processing.py. The txt2img() function in modules/txt2img.py builds this object from the Gradio UI inputs and passes it through the script pipeline.

Step	What Happens
1. Build processing object	txt2img_create_processing() constructs StableDiffusionProcessingTxt2Img with prompt, negative_prompt, cfg_scale, width, height, sampler, steps, seed, batch_size, highres fix params, etc.
2. Script pre-processing	scripts_txt2img.run(p, *args) — each Script.run() can modify p or short-circuit with a Processed result
3. process_images(p)	Core pipeline: parse prompts, create latent noise, run denoising loop via chosen sampler
4. Prompt parsing	prompt_parser.get_learned_conditioning() processes attention weights, scheduled swaps, AND compositions into per-step conditioning tensors
5. Latent noise init	rng.ImageRNG generates reproducible noise tensors (Philox or CPU RNG depending on settings)
6. Denoising loop	Sampler steps through the diffusion schedule; CFG denoiser applies classifier-free guidance at each step
7. VAE decode	Latent tensor decoded to pixel space via the model's VAE (or TAESD/approx VAE for live previews)
8. Highres Fix (if enabled)	Image upscaled (via selected hr_upscaler), then a second denoising pass refines the upscaled image
9. Post-processing scripts	Each Script.postprocess_image() called — face restoration, custom upscaling, etc.
10. Image save	images.save_image() writes output with metadata embedded in PNG chunks or JPEG EXIF

3.2 Img2Img Pipeline

Image-to-image processing is handled by StableDiffusionProcessingImg2Img. The img2img() function in modules/img2img.py resolves one of five input modes before creating the processing object.

Mode	Value	Input Sources	Mask
Standard img2img	0	init_img	None
Sketch	1	sketch (drawing canvas)	None

Inpaint	2	init_img_with_mask['image']	init_img_with_mask['mask'] — binary
Inpaint Sketch	3	inpaint_color_sketch (diff from original)	Derived from color difference + blur
Inpaint Upload Mask	4	init_img_inpaint	init_mask_inpaint (uploaded)
Batch	5	Directory path or file upload list	Optional inpaint_mask_dir for batch inpaint

The img2img pipeline encodes the init_image into the latent space (rather than starting from pure noise), then runs denoising at the strength specified by denoising_strength (0.0 = no change, 1.0 = completely ignore the init image). The masking pipeline in modules/masking.py handles blending the masked vs unmasked regions.

3.3 Sampler System

Samplers are the algorithms that iteratively denoise the latent space. The webui unifies multiple sampler families under a common SamplerData interface in modules/sd_samplers_common.py.

Sampler Family	Module	Examples
K-Diffusion	sd_samplers_kdiffusion.py	Euler, Euler a, DPM++ 2M, DPM++ SDE, DDPM, Heun, LMS, DPM2, DPM2 a, DPM++ 2S a
CompVis Timesteps	sd_samplers_timesteps.py	DDIM, PLMS, UniPC
LCM	sd_samplers_lcm.py	LCM (Latent Consistency Models — very fast, few steps)
Schedulers	sd_schedulers.py	Linear, Cosine, Karras, Exponential, SGM Uniform, Beta

3.4 Model Support

Model Type	Enum	Notes
Stable Diffusion 1.x	ModelType.SD1	Original CompVis architecture; CLIP text encoder; v1-inference.yaml config
Stable Diffusion 2.x	ModelType.SD2	OpenCLIP text encoder; 768x768 default resolution
Stable Diffusion XL	ModelType.SDXL	Dual text encoders (CLIP-L + OpenCLIP-G); 1024x1024 default; handled by sd_models_xl.py
SSD-1B	ModelType.SSD	Segmind Stable Diffusion distilled SDXL variant
Stable Diffusion 3	ModelType.SD3	SD3 architecture; sd3-inference.yaml config

Alt-Diffusion	—	Multilingual CLIP; alt-diffusion-inference.yaml / m18 variant
InstructPix2Pix	—	Image editing via instructions; instruct-pix2pix.yaml
Inpainting models	—	RunwayML dedicated inpaint checkpoints; v1-inpainting-inference.yaml

4. Prompt System

The prompt parser in modules/prompt_parser.py uses a Lark grammar to parse an extended prompt syntax well beyond plain text. This is one of the most powerful features of the WebUI.

4.1 Prompt Syntax Reference

Syntax	Example	Effect
Attention weight	(word:1.3)	Increases model attention to 'word' by factor 1.3
Increased attention	((word))	Each () pair multiplies weight by 1.1
Decreased attention	[word]	Multiplies weight by 0.9 (weakens)
Prompt scheduling	[mountain:lake:0.5]	Use 'mountain' for first 50% of steps, then 'lake'
Prompt scheduling (add)	[: in winter :0.7]	Add 'in winter' from 70% of steps onward
Alternating prompts	[cat|dog]	Alternates between 'cat' and 'dog' each step
Prompt composition	a cat AND a dog :2.0	Compose multiple prompts; weights control influence
No token limit	(unlimited prompt length)	WebUI bypasses the original 75-token CLIP limit via chunking
Textual Inversion ref	<embedding_name>	Injects a trained embedding into the conditioning
Lora activation	<lora:model_name:0.8>	Applies a LoRA adapter with given weight
Hypernetwork	<hypernet:name:strength>	Applies a hypernetwork at the given strength

4.2 Extra Networks (LoRA / Hypernetworks / Embeddings)

Extra networks are model add-ons referenced inline in the prompt. The modules/extra_networks.py framework parses extra network syntax from prompts, activates the corresponding adapter or embedding during the conditioning phase, and restores the model afterward. The UI provides a visual browser panel (Extra Networks tab) showing thumbnails of all available LoRAs, hypernetworks, and embeddings with click-to-insert functionality.

5. Scripts & Extension System

5.1 Built-in Scripts

Script	File	What It Does
X/Y/Z Grid	xyz_grid.py	Renders a grid of images varying up to 3 parameters (sampler, CFG, steps, seed, prompt, checkpoint, etc.)
Prompt Matrix	prompt_matrix.py	Enumerates all combinations of comma-separated prompt segments as a grid
Prompts from File	prompts_from_file.py	Reads prompts line-by-line from a text file for batch generation
Loopback	loopback.py	Feeds each generated image back as input for N iterations
SD Upscale	sd_upscale.py	Tile-based upscaling: upscales the image then re-renders each tile with img2img
Outpainting MK2	outpainting_mk_2.py	Extends the canvas in any direction using overlapping tiles
Poor Man's Outpaint	poor_mans_outpainting.py	Simpler outpainting via padding and inpainting
Img2Img Alternative	img2imgalt.py	Reverse Euler cross-attention control for more faithful img2img
Custom Code	custom_code.py	Executes arbitrary Python code (requires --allow-code flag)
GFPGAN Postprocess	postprocessing_gfpgan.py	Face restoration as a postprocessing step
CodeFormer Postprocess	postprocessing_codeformer.py	CodeFormer face restoration as a postprocessing step
Upscale Postprocess	postprocessing_upscale.py	Upscaling as a postprocessing step in the Extras pipeline

5.2 Script Base Class

All scripts (both built-in and from extensions) inherit from modules/scripts.py's Script class. Key methods that scripts can implement:

- title(): Return the script's display name in the UI dropdown.
- show(is_img2img): Return True/False/AlwaysVisible to control when the script appears.
- ui(is_img2img): Build and return Gradio components for script-specific settings.
- run(p, *args): Main script entry point. Receives the processing object p and script UI values; can return a Processed result to short-circuit the default pipeline.

- postprocess(p, processed, *args): Called after all images are generated; can modify the Processed result.
- postprocess_image(p, pp, *args): Called once per generated image for per-image post-processing.

5.3 Callback / Hook System

modules/script_callbacks.py provides a named event system with 30+ hooks. Extensions register functions against these hooks to intercept specific moments in the processing lifecycle without needing to subclass or patch core code.

Callback	When It Fires
on_app_started	After the Gradio app and FastAPI app are fully started
on_before_ui	Just before create_ui() is called; good for registering tabs
on_ui_tabs	When extra tabs are being collected for the main UI
on_ui_settings	When the Settings tab is being built; add custom settings sections here
on_after_component	After every Gradio component is created; allows injecting UI modifications
on_image_saved	After each image is saved to disk; receives file path and generation info
on_script_unloaded	When the webui is restarting; clean up resources
on_list_optimizers	When cross-attention optimizers are being registered
on_list_unets	When UNet implementations are being registered
on_cfg_denoiser_forward	At every CFG denoiser forward pass — deepest processing hook
on_model_loaded	After a checkpoint finishes loading

5.4 Extension Architecture

Extensions are git repositories placed in the extensions/ directory. Each extension can contain scripts/ (loaded like built-in scripts), javascript/ (loaded into the page), localizations/ (translation files), style.css, and an install.py for custom setup steps. The Extensions tab in the UI provides one-click install from a curated list, update management, and enable/disable controls. The loading order is determined by a topological sort of declared dependencies.

6. REST API

When launched with --api (alongside the UI) or --nowebui (headless API-only mode), the webui exposes a full FastAPI REST interface documented at /docs (Swagger) and /redoc. Every UI capability has an API equivalent, enabling automation pipelines, remote generation, and third-party tool integration.

6.1 Key API Endpoints

Endpoint	Method	Purpose
/sdapi/v1/txt2img	POST	Generate images from text prompt; returns base64-encoded images
/sdapi/v1/img2img	POST	Image-to-image transformation with all 5 modes
/sdapi/v1/extra-single-image	POST	Upscale or restore a single image via Extras pipeline
/sdapi/v1/extra-batch-images	POST	Batch upscale/restore images
/sdapi/v1/png-info	POST	Extract generation parameters embedded in a PNG/JPEG
/sdapi/v1/progress	GET	Poll current generation progress (step, ETA, preview image)
/sdapi/v1/interrupt	POST	Interrupt the current generation
/sdapi/v1/skip	POST	Skip the current image in a batch
/sdapi/v1/options	GET	Get all current settings
/sdapi/v1/options	POST	Update settings at runtime
/sdapi/v1/cmd-flags	GET	Get all command-line flags
/sdapi/v1/samplers	GET	List all available samplers
/sdapi/v1/sd-models	GET	List all available checkpoints
/sdapi/v1/sd-vae	GET	List all available VAE files
/sdapi/v1/loras	GET	List available LoRA models
/sdapi/v1/embeddings	GET	List available textual inversion embeddings
/sdapi/v1/hypernetworks	GET	List available hypernetworks
/sdapi/v1/upscalers	GET	List available upscaler models
/sdapi/v1/face-restorers	GET	List available face restoration models
/sdapi/v1/memory	GET	Get current RAM and VRAM usage
/sdapi/v1/train/textual-inversion	POST	Start a textual inversion training run
/sdapi/v1/train/hypernetwork	POST	Start a hypernetwork training run
/sdapi/v1/server-kill	POST	Kill the server (requires --api-server-stop)

/sdapi/v1/server-restart	POST	Restart the server (requires --api-server-stop)

6.2 API Authentication

API authentication is controlled by --api-auth "username:password" (comma-separated for multiple users). Optionally, TLS can be enabled with --tls-keyfile and --tls-certfile. CORS origins are configurable via --cors-allow-origins or --cors-allow-origins-regex. By default (without --api-auth), the API is open to all requests on the bound network interface.

7. Hardware & VRAM Management

7.1 Hardware Backends

Backend	Flag / Condition	Notes
NVIDIA CUDA	Default when CUDA available	Full feature support; recommended hardware
AMD ROCm	Install ROCm PyTorch build	Linux only; most features supported
Apple MPS	webui-macos-env.sh	macOS Apple Silicon; some operations fall back to CPU
Intel XPU	--use-ipex	Intel Arc GPUs via Intel Extension for PyTorch
Ascend NPU	requirements_npu.txt	Huawei Ascend hardware via CANN toolkit
CPU only	--use-cpu all	Slow but works on any machine; float32 only

7.2 VRAM Optimization Modes

Mode	Flag	Minimum VRAM	Mechanism
Full precision	(none)	8+ GB	Model stays in VRAM at fp16/autocast
Medium VRAM	--medvram	4+ GB	cond/uncond batching disabled; some offloading
Medium VRAM XL	--medvram-sdxl	6+ GB	--medvram applied only for SDXL models
Low VRAM	--lowvram	2+ GB	Module-by-module CPU offloading via lowvram.py
Low RAM	--lowram	—	Load checkpoint weights directly to VRAM
No half	--no-half	16+ GB	fp32 throughout; for GPUs that struggle with fp16
No half VAE	--no-half-vae	—	fp32 VAE only; fixes black image issues on some GPUs
Upcast sampling	--upcast-sampling	—	Upcasts sampling ops to fp32 for stability

7.3 Cross-Attention Optimizations

The attention computation in the UNet is the most VRAM-intensive operation. The webui supports multiple optimization implementations selectable via command-line flags:

- Scaled Dot Product Attention (--opt-sdp-attention): PyTorch 2.0+ built-in; best for most modern GPUs.
- xformers (--xformers): Facebook Research's memory-efficient attention; major speedup on NVIDIA cards.
- Doggettx split attention (--opt-split-attention): Chunked attention for older GPUs.
- Sub-quadratic attention (--opt-sub-quad-attention): Memory-efficient alternative.
- InvokeAI attention (--opt-split-attention-invokeai): Alternative split-attention implementation.
- No optimization (--disable-opt-split-attention): Baseline, uses most VRAM.
- Token merging (tomesd): Merges similar tokens to reduce sequence length; configurable ratio in Settings.

8. Model Hijacking & Monkey-Patching

A distinctive architectural feature of the WebUI is its use of Python monkey-patching (the 'hijack' modules) to modify the behavior of the loaded Stable Diffusion model at runtime without forking the upstream model code. This allows the UI to stay compatible with upstream changes while adding capabilities.

Module	What It Patches / Adds
sd_hijack.py	Main hijack entry point: applies CLIP hijack, attention optimizations, and any registered optimizer
sd_hijack_clip.py	Replaces CLIPTextModel's forward method to: remove 75-token limit, support per-chunk encoding, inject learned embeddings
sd_hijack_open_clip.py	Same treatment for OpenCLIP (used by SD2.x and SDXL)
sd_hijack_optimizations.py	Registers all cross-attention optimization implementations; patches attention modules with selected optimizer
sd_hijack_unet.py	Patches UNet forward to support unet override system and profiling hooks
sd_hijack_checkpoint.py	Patches model checkpoint loading for safe unpickling and custom loading behavior
sd_hijack_ip2p.py	Patches InstructPix2Pix (image-cfg-scale conditioning)
sd_hijack_xlmr.py	Alt-Diffusion multilingual text encoder hijack
patches.py	Generic undo-able patch utility used by all hijack modules
sd_unet.py	UNet override registry: extensions can register alternative UNet implementations
sd_models_xl.py	SDXL-specific model loading and conditioning patches

9. Upscalers & Post-Processing

9.1 Available Upscalers

Upscaler	Module	Notes
Lanczos	upscaler.py (built-in)	PIL Lanczos — no neural network; fast, baseline quality
Nearest	upscaler.py (built-in)	PIL Nearest — pixel-perfect scaling; for pixel art
ESRGAN	esrgan_model.py	General-purpose super-resolution; many community model variants
RealESRGAN	realesrgan_model.py	Improved ESRGAN trained on real-world degraded images
SwinIR	(via spandrel)	Transformer-based SR; high quality
Swin2SR	(via spandrel)	Successor to SwinIR
HAT	hat_model.py	Hybrid Attention Transformer; high quality SR
DAT	dat_model.py	Dual Aggregation Transformer upscaler
LDSR	(external repo)	Latent Diffusion Super Resolution; slowest, highest quality
4x-UltraSharp	(community .pth file)	Popular ESRGAN community model; place in models/ESRGAN/

9.2 Face Restoration

Tool	Module	Notes
GFPGAN	gfpgan_model.py	GAN-based face restoration; fast; requires GFPGAN model in models/GFPGAN/
CodeFormer	codeformer_model.py	Transformer-based face restoration; generally higher fidelity than GFPGAN; fidelity weight configurable

9.3 Extras Pipeline

The Extras tab (modules/extras.py) provides a standalone post-processing pipeline separate from generation. Users can upscale any image (with optional face restoration), batch-process a directory, or post-process generation outputs. The pipeline applies upscaling first, then face restoration, or both in a configurable order.

10. Settings & Configuration

10.1 Command-Line Arguments (Key Selections)

Argument	Default	Purpose
--ckpt	v1-5-pruned.ckpt	Path to the SD checkpoint to load at startup
--ckpt-dir	models/Stable-diffusion/	Directory scanned for all available checkpoints
--vae-dir	models/VAE/	Directory for VAE files
--embeddings-dir	embeddings/	Directory for textual inversion embeddings
--port	7860	HTTP port for the Gradio server
--listen	False	Bind to 0.0.0.0 (expose to network); off by default
--share	False	Create a public Gradio share URL via gradio.live
--api	False	Enable the REST API alongside the UI
--nowebui	False	Run only the API, no Gradio UI
--xformers	False	Enable xformers cross-attention optimization
--medvram	False	Medium VRAM optimization mode
--lowvram	False	Low VRAM optimization mode
--precision	autocast	Numeric precision: full / half / autocast
--no-half	False	Force fp32 (no fp16)
--gradio-auth	None	"user:pass" basic auth for the web UI
--api-auth	None	"user:pass" basic auth for the API
--allow-code	False	Allow executing Python code from UI (security risk)
--disable-safe-unpickle	False	Skip malicious code check on .ckpt files (unsafe)
--autolaunch	False	Open browser automatically on startup
--subpath		URL subpath for reverse proxy deployments
--cors-allow-origins	None	Comma-separated list of allowed CORS origins for the API

10.2 Runtime Settings (UI Settings Tab)

The Settings tab (built by modules/ui_settings.py from options defined in modules/shared_options.py) provides hundreds of adjustable parameters that take effect without restart, including: output directories and filename patterns, saving formats (PNG/JPEG/WebP/AVIF) and quality, sampler defaults, Highres Fix defaults, face restoration settings, upscaler tile sizes, Extra Networks display options, UI theme and reordering, and model-specific options like CLIP skip and token merging ratio.

11. Implementation Approach

11.1 Prerequisites

- Python 3.10.6 (Windows) or Python 3.10.x (Linux/macOS). Python 3.11+ may work but is not officially tested.
- git — required for cloning the repository and installing extensions.
- NVIDIA GPU with 4 GB+ VRAM recommended; 2 GB possible with --lowvram. AMD/Apple/Intel supported with extra steps.
- Stable Diffusion checkpoint file (.safetensors or .ckpt) — not included; must be downloaded separately from HuggingFace or CivitAI.

11.2 Installation — Windows (NVIDIA)

Step	Action
1	Install Python 3.10.6 from python.org. Check 'Add Python to PATH'.
2	Install git from git-scm.com.
3	Clone or download the repository: git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git
4	Place your SD checkpoint (.safetensors) in models/Stable-diffusion/
5	Run webui-user.bat — it creates a venv, installs all dependencies, and starts the server.
6	Browse to http://127.0.0.1:7860

11.3 Installation — Linux (NVIDIA)

```
# Install system dependencies
sudo apt install wget git python3 python3-venv libgl1 libglib2.0-0

# Clone repository
git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git
cd stable-diffusion-webui

# Place checkpoint in models/Stable-diffusion/
# Then launch:
./webui.sh
```

11.4 Customising the Launch (webui-user.sh / webui-user.bat)

The recommended way to set persistent launch flags is via webui-user.sh (Linux/Mac) or webui-user.bat (Windows). Set the COMMANDLINE_ARGS variable:

```
# webui-user.sh example
export COMMANDLINE_ARGS="--xformers --medvram --api --port 7860 --listen"
```

Common Flag Combinations	Use Case

--xformers	Best performance on NVIDIA GPUs with sufficient VRAM
--medvram --xformers	4-6 GB VRAM cards
--lowvram	2-4 GB VRAM (very slow)
--api	Enable REST API for automation
--listen --port 7860	Expose to local network (e.g. access from another PC)
--share	Create temporary public URL via Gradio tunnel
--gradio-auth user:pass	Password-protect the web UI
--no-half --precision full	Force fp32 for GPUs with fp16 issues
--subpath webui	Run behind a reverse proxy at /webui/
--disable-all-extensions	Debug mode: disable all extensions

11.5 Installing a Model

- Download a Stable Diffusion checkpoint from Hugging Face (e.g. stabilityai/stable-diffusion-xl-base-1.0) or CivitAI.
- Place .safetensors or .ckpt files in models/Stable-diffusion/. Subdirectory organisation is supported.
- Click the refresh button next to the checkpoint dropdown in the UI, or simply restart.
- To use a separate VAE, place .safetensors/.pt VAE files in models/VAE/ and select them in Settings > Stable Diffusion > SD VAE.

11.6 Installing Extensions

Extensions are the primary way to add new features. Three installation methods:

- Via UI: Open Extensions tab > Install from URL, paste the extension's git URL, click Install, then Apply and restart UI.
- Via Available tab: Extensions tab > Available > Load from (loads the official extension index), click Install next to any extension.
- Manually: git clone the extension repository into extensions/ directory, then restart the UI.

11.7 Deployment Considerations

- Security: Do not expose the UI to the public internet without --gradio-auth. The API should use --api-auth. The --allow-code flag enables arbitrary code execution — never use in untrusted environments.
- CORS: Set --cors-allow-origins to specific origins if the API will be called from a browser application.
- Reverse Proxy: Use --subpath if running behind nginx/Caddy. Configure the proxy to forward WebSocket connections (Gradio uses WebSockets for the queue).

- GPU Sharing: Multiple users share one GPU; the Gradio queue (capacity 64 by default) serializes requests. For multi-GPU, run multiple instances on different ports.
- Persistent Config: Use webui-user.sh/bat for all persistent flags. Runtime settings are saved to config.json (editable via the Settings tab).
- Updates: git pull in the repository directory, then restart. Extension updates available via the Extensions tab.

11.8 Adding a Custom Script

To add a custom script, create a .py file in the scripts/ directory. It must define a class inheriting from modules.scripts.Script:

```
import modules.scripts as scripts
import gradio as gr

class MyScript(scripts.Script):
    def title(self):
        return 'My Custom Script'

    def show(self, is_img2img):
        return True  # or scripts.AlwaysVisible

    def ui(self, is_img2img):
        my_slider = gr.Slider(label='My Param', value=0.5, minimum=0, maximum=1)
        return [my_slider]

    def run(self, p, my_value):
        # modify p or call processing.process_images(p)
        p.extra_generation_params['my_param'] = my_value
        return None  # None = use default pipeline
```

12. Summary

Stable Diffusion WebUI is a mature, production-quality open-source project that serves as the reference implementation for local Stable Diffusion inference. Its architecture cleverly combines a Gradio browser UI with a FastAPI REST backend, unified by a shared processing pipeline that is fully extensible at every stage. The hijack/monkey-patch pattern allows it to augment upstream model libraries without forking them, and the script/callback system enables a large extension ecosystem with minimal core changes.

The primary complexity lies in the breadth of features and hardware targets rather than deep algorithmic novelty. For a new developer, the entry points are: webui.py (startup), modules/processing.py (diffusion pipeline), modules/scripts.py (extension points), and modules/api/api.py (REST interface).

Feature Area	Status	Key Files
Txt2Img generation	Complete	modules/txt2img.py, processing.py
Img2Img (5 modes)	Complete	modules/img2img.py, processing.py, masking.py
Highres Fix	Complete	txt2img.py (two-pass logic in processing.py)
Inpainting / Outpainting	Complete	img2img.py, masking.py, scripts/outpainting*.py
Checkpoint support (SD1-SD3)	Complete	modules/sd_models.py, sd_models_xl.py
VAE hot-swap	Complete	modules/sd_vae.py
Sampler system	Complete	modules/sd_samplers*.py
Prompt scheduling & attention	Complete	modules/prompt_parser.py
LoRA / Hypernetwork / Embeds	Complete	modules/extra_networks*.py
Upscalers (7+ models)	Complete	modules/upscaler.py, *_model.py, spandrel
Face restoration (GFPGAN/CF)	Complete	modules/gfpgan_model.py, codeformer_model.py
REST API	Complete	modules/api/api.py
Extension system	Complete	modules/extensions.py, scripts.py, script_callbacks.py
Built-in scripts (12)	Complete	scripts/*.py
Training (TI / Hypernet)	Complete	modules/textual_inversion/
Multi-hardware (CUDA/AMD/MPS/Intel/NPU)	Complete	modules/devices.py, *_specific.py
VRAM optimization modes	Complete	modules/lowvram.py, devices.py

xformers / SDP attention	Complete	modules/sd_hijack_optimizations.py
Localization / i18n	Complete	modules/localization.py, localizations/
AVIF image format support	Complete	modules/images.py via pillow-avif-plugin